

ECE 160 Final Project

snakECE160

"Snake!"



Presented by

Felix B

Alan L

Achilles K

Content

1 Snake Core



2 Effects

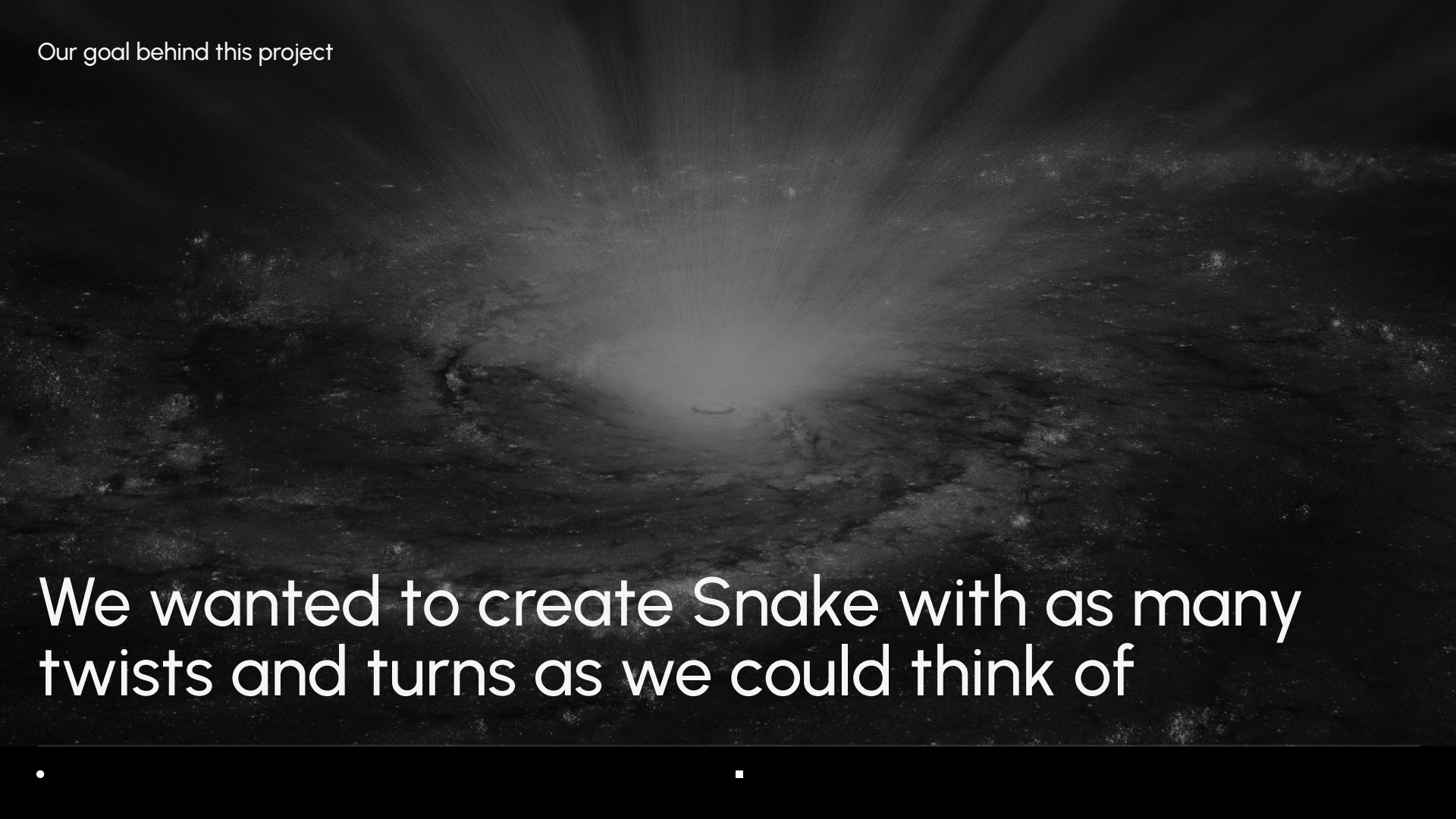


3 QOL Features



4 Bug Fixes





Our goal behind this project

We wanted to create Snake with as many twists and turns as we could think of

GameObject

```
class GameObject:
    objList = []
    def __init__(self, pos):
        if type(self) not in (PopUps, musicBox):
            GameObject.objList.insert(0, self)
        else:
            GameObject.objList.append(self) ## popup needs to render last, and we can guarantee that by adding it into the last
        self.pos = pos if pos else [0, 0] #yeah
        self.pos.append(0) if len(self.pos) == 2 else 0 ## 3d coordinates, from base.
        self.color = 'magenta' #if anything is magenta colored, that means it has been setup invalidly. warning color.
        self.rect = pg.rect.Rect(CELL_LENGTH*self.pos[0], CELL_HEIGHT*self.pos[1], CELL_LENGTH, CELL_HEIGHT) # rect anchored to cell Q2 corner, with dimensions of a single cell.

    def render(self, screenV=screen):
        pg.draw.rect(screenV, self.color, self.rect)

    def collide(self, snake):
        pass # default collision means nothing

    @classmethod
    def Render(cls, screenV=screen):
        for obj in cls.objList:
            obj.render(screenV)

    @classmethod
    def Collide(cls, snake):
        for obj in [x for x in cls.objList if x != snake]:
            #print(type(obj), obj)
            snake.collide(obj)

    @classmethod
    def Reset(cls): ## works for each class extends GameObject (ex. Snake, Apple, PopUps, )
        for i in [x for x in GameObject.objList if type(x)==cls]:
            GameObject.objList.remove(i)
```

```
def snakeGame(menu, snake, progress): ## this is the actual main game loop function!! yay
    run = True
    while run:
        progress.check(snake.len)
        bgSwitch(snake.len)
        keysPressed = pg.key.get_pressed()

        if keysPressed[pg.K_ESCAPE]: ## this is up here to break before anything else--
            for event in pg.event.get(): --
            if keysPressed[pg.K_ESCAPE]: --

        snake.steer(keysPressed) # alter the snake movement vector based on keypresses.
        GameObject.Render(screen) # to be clear, renders all game objects.
        pg.display.flip() # update screen!
        clock.tick(framerate) # limit framerate to framerate
```

GameObject is the parent class of all interactive objects, and as a result, has most of the foundational elements needed to permit rendering, collision, and resets. Subclasses of GameObject that have special rendering/collide requirements/bonus logic overload the Render method, as seen in PopUp, Apple, MusicBox, etc etc...

```
class Clickable(GameObject):
    def __init__(self, pos=None):
        super().__init__(pos)
        self.last_tick = pg.time.get_ticks()
    def clicked(self):
        return self.rect.collidepoint(pg.mouse.get_pos()) and (pg.mouse.get_pressed())[0]
```

Snake

```
class Snake(GameObject):
    _instance = None ## ok frankly this was done by ai. I have YET to understand this chunk, but it will come soon.
    def __new__(cls, *args, **kwargs):
        if cls._instance is not None:
            cls._instance.__del__() # Clean up old instance from GameObject.objList
            cls._instance = super().__new__(cls)
            return cls._instance
        # ai code over
    def __init__(self, sMat, **args): #PLEASE pass custompos as a 3 val array if it is being used.
        self.sMat = sMat ## reset. lazy. oh well.
        initpos = sMat.center[:] if (not ('startPos' in args)) else args['startPos']
        super().__init__(initpos) #sets self.pos
        self.cols, self.rows = sMat.cols, sMat.rows # begrudging.
        self.color = 'green'
        self.len = 1 ## inital length!!! very important

        self.direction = pg.Vector3(1,0,0) # three-dimensional movement possibilites. also start by moving right to avoid self collision at beginning
        self.specialFlags = {} # for custom controls-ish

        if 'controls' in args.keys(): # ALL OF THESE HAVE TO EXIST OR THE PROGRAM DIES.
            self.up = args['controls']['up']
            self.down = args['controls']['down']
            self.left = args['controls']['left']
            self.right = args['controls']['right']
            self.interact = args['controls']['interact']
            self.accel = args['controls']['accel']
            self.decel = args['controls']['decel']
        else: #default case... jacinthe's theme slaps so hard.
            self.up = pg.K_w
            self.down = pg.K_s
            self.left = pg.K_a
            self.right = pg.K_d
            self.interact = pg.K_e
```

Snake inherits GameObject, and is our core player class. It sets all relevant constants, and is the first instance of a monoclase that is used in the code. This ensures there aren't any duplicate Snake classes (we aren't making Tron).

The Snake is the object initiating collision with other objects, so it makes sense to have collision initiated in the Snake class. As a result, to get the Snake collision with an object, `GameObject.Collide()` is called, leading to `snake.collide()` being called for each `GameObject`.

```
def futurePos(self):
    ## basically, this returns coordinates. It's the movement function, but doesn't update the movement
    return [(self.pos[0] + int(self.direction.x)) % self.cols,
            (self.pos[1] + int(self.direction.y)) % self.rows,
            (self.pos[2] + int(self.direction.z))]

def collide(self, obj):
    #print(type(self), type(obj))
    if self.futurePos() == obj.pos:
        obj.collide(self)

def move(self):
    self.pos = self.futurePos()[:]
    self.rect[0], self.rect[1] = (CELL_LENGTH*self.pos[0]), (CELL_HEIGHT*self.pos[1])#rect updater
    SnakeTail.tailList.insert(0, SnakeTail(self.pos[:])) ## insert the new head position at the start of the list
    while len(SnakeTail.tailList) > self.len: # needs to be while, for all the times when i trim the snake more than 1 unit
        SnakeTail.Sever() ## remove tail if array is bigger than length
    if self.specialFlags.get("debugPrint", False): # we can actually adapt this system for like a boost or whatever.
        Apple.Reset()
        self.specialFlags["debugPrint"] = False # since inputs are processed 24/7, this allows a certain action to be queued, then happen when the snake moves. Works well!

def steer(self, keys):
    if keys[self.up]:
        self.direction = pg.Vector3(0,-1,0) if self.direction.y != 1 else self.direction
    elif keys[self.down]:
        self.direction = pg.Vector3(0,1,0) if self.direction.y != -1 else self.direction
    elif keys[self.left]:
        self.direction = pg.Vector3(-1,0,0) if self.direction.x != 1 else self.direction
    elif keys[self.right]:
        self.direction = pg.Vector3(1,0,0) if self.direction.x != -1 else self.direction
    if keys[self.interact]:
        self.specialFlags["debugPrint"] = True # removed length increase cuz jank, did i mess up array?
```

SnakeTail

```
class SnakeTail(GameObject):
    tailList = []
    def __init__(self, pos=[0,0,0]):
        super().__init__(pos)
        self.color = 'yellow'

    def collide(self, snake):
        pg.event.post(game_over)

    @classmethod
    def Sever(cls):
        GameObject.objList.remove(cls.tailList.pop())
    def Reset(cls):
        cls.tailList = []
```

The tail of the snake used to be non-collidable and rendered using a separate list, but now it uses my beloved GameObject class!

In particular, the tail has a Sever method, which only removes the last segment from the Snake tail. This is how the tail should work, and follows a FIFO structure. As a result, the Reset method for SnakeTail resets that list, instead of objects known as SnakeTail (same effect tho)

Main Game Loop

```
def snakeGame(menu, snake, progress): ## this is the actual main game loop function!! yay
    run = True
    while run:
        progress.check(snake.len)
        bgSwitch(snake.len)
        keysPressed = pg.key.get_pressed()

        if keysPressed[pg.K_ESCAPE]: ## this is up here to break before anything else...
            for event in pg.event.get():
                if event.type == pg.QUIT:
                    pg.quit() # GET OUT!!!!
                    exit(0)
                if event.type == SNAKE_EVENT:
                    GameObject.Collide(snake) # collide objects with snake before moving it
                    ## create objects
                    if progress.contain("popup"): ~
                    if progress.contain("music_box"): ~
                    if keysPressed[pg.K_RETURN]: ~
                    snake.move() # then move snake afterwards.
                    if (random.randint(0, 120) < 1) and (not QTE._instance) and ("QTE" in progress.unlocked):
                        QTE()
                if event.type == GAMEOVER:
                    GameObject.Reset()
                    gameover()
                    gos = GameOverScreen(screen=screen, clock=clock, win_h=SCREEN_HEIGHT, win_w=SCREEN_WIDTH,
                    gos.run()
                    run = False

        snake.steer(keysPressed) # alter the snake movement vector based on keypresses.
        GameObject.Render(screen) # to be clear, renders all game objects.
        pg.display.flip() # update screen!
        clock.tick(framerate) # limit framerate to framerate
    menu.notstop = True
```

- SNAKE_EVENT lets us cap the actual processing of game events – basically, SNAKE_EVENT runs every 67 ms, instead of every frame (similar to delta time, but using pygame events instead of subtraction and time)
- We check progress, set background accordingly, then get keys pressed, then in the SNAKE EVENT loop:
 - Collide first
 - Spawn new stuff second
 - Move the snake in the direction
- If Gameover, reset
- Then, out of the SNAKE_EVENT loop:
 - Steer snake – allows to change direction at anytime, instead of just when the snake updates
 - Render all objects
 - Flip display
 - Tick clock

MusicBox

How it works

- Timer
- Gathers clicking location
- Own timers
 - When holding down left click on the box, every tick it adds a certain amount to the total time remaining
 - Else it will decrease every tick

```
class musicBox(Clickable): 4 usages & Felix +2
    maxTime = 1200
    def __init__(self, pos=[((COLUMN_COUNT//2)+(COLUMN_COUNT//4)),((ROW_COUNT//2)+(ROW_COUNT//4)),0]):
        super().__init__(pos)
        self.rect.scale_by_ip(9,7)
        self.time = musicBox.maxTime
        self.color = pg.color.Color(0, 0, 255, 76)
        #self.last_tick = pg.time.get_ticks() but in Clickable
    def render(self, screenV = screen): 1 usage (1 dynamic) & Felix +2
        #I need this to be translucent lmao
        current_time = pg.time.get_ticks()
        if (current_time - self.last_tick >= 1000) and not self.clicked():
            self.time -= 60 ## tick down by a second, only when not clicked tho
            self.last_tick = current_time
        super().render(screenV) # render box to screen

        text = font.render(text=f"Time: {self.time // 60}", antialias=True, color=(255, 255, 255))
        screenV.blit(text, dest=(self.rect.x + 10, self.rect.y + 15)) # render text to screen

        if self.clicked() and self.time < musicBox.maxTime:
            self.time += 2 #slow wind
        if self.time <= 0:
            pg.event.post(game_over) # faahhhhhh
```

Popups

How it works

- Timer
- Gathers clicking location
- Own timers
 - Every tick is scales to be bigger
 - It scales by a certain k every tick
 - Scale time speeds up
 - Once it gets to a certain size, it deletes itself and sets the snake length to 1
 - If clicked, it pops itself and respawns

```
class PopUps(Clickable): 4 usages 2 Felix +1
    def __init__(self, pos=None): 2 Felix +1
        super().__init__((pos or [random.randint(a: 0,COLUMN_COUNT),random.randint(a: 0,ROW_COUNT)]))
        self.color = popUpColor
        self.rect.scale_by_ip(3.7,3.7)
        self.timeScale = 600

def render(self, screenV=screen): 1 usage (1 dynamic) 2 Felix +1
    if pg.time.get_ticks() - self.last_tick >= self.timeScale:
        self.rect.scale_by_ip(1.2, 1.2)
        self.last_tick = pg.time.get_ticks()
        self.timeScale //= 1.2
        if self.timeScale == 1:
            GameObject.objList.remove(self)
            for i in [x for x in GameObject.objList if type(x) == Snake]:
                i.len = 1 #the Fool
    if self.clicked():
        GameObject.objList.remove(self)
        PopUps()
    super().render(screenV)
```

QTE

```
class QTE(Clickable): 5 usages 2 ToastPixel +1
    _instance = None
    def __init__(self, pos=None, key=pg.K_SPACE, duration=2400): 2 ToastPixel +1
        QTE._instance = self
        super().__init__(pos or [random.randint((COLUMN_COUNT//10), COLUMN_COUNT - (COLUMN_COUNT//10)), random.randint(0, 200), 255])
        self.color = (0, 200, 255)
        self.rect.scale_by_ip(17, 4)
        self.key = key
        self.max = duration
        self.duration = duration
        self.start_time = pg.time.get_ticks()

    def render(self, screenV=screen): 1 usage (1 dynamic) 2 ToastPixel +1
        current_time = pg.time.get_ticks()
        elapsed = current_time - self.start_time

        if elapsed >= self.max:
            self.active = False
            print("QTE failed")
            pg.event.post(game_over)
            return False

        # Check for successful key press
        if pg.key.get_pressed()[self.key]:
            GameObject.objList.remove(self)
            QTE._instance = None ## required for respawn enable
            print("QTE success")
            return

        super().render(screenV)
        txt = font.render(text=f"PRESS SPACE! {elapsed}", antialias=True, color=(255, 255, 255), )
        screenV.blit(txt, dest=(self.rect.x + 5, self.rect.y + 5))
```

How it works

- Similar to music box
- It calculates the current time elapsed
 - Constantly checks if space is checked
 - If the time elapsed passes the duration set, it will throw a gameover
 - It also uses render to display the screen

Progression

```
class Progression(): 2 usages 2 ToastPixel +1
    _instance = None
    def __init__(self): 2 ToastPixel +1
        Progression._instance = self
        self.lvls = {
            10: "popup",
            3: "QTE",
            17: "music_box",
            25: "popup",
        }
        self.unlocked = []
        self.activated = []

    def check(self, length): 1 usage (1 dynamic) 2 ToastPixel
        newly_unlocked = []

        for n, f in self.lvls.items():
            if length >= n:
                if not self.unlocked.__contains__(f):
                    print(f"unlocked {f}")
                    self.unlocked.append(f)
                    self.activated.append(f)
                    newly_unlocked.append(f)
        return newly_unlocked

    def contain(self, name): 2 usages (2 dynamic) 2 ToastPixel
        if name in self.unlocked and name in self.activated:
            self.activated.remove(name)
            return True
        else:
            return False

@classmethod 2 Felix
```

```
        return False

@classmethod 2 Felix
def Reset(cls):
    for i in cls.lvls.values():
        if i in cls.unlocked:
            cls.unlocked.remove(i)
        if i in cls.activated:
            cls.activated.remove(i)

class SnakeMat: 1 usage 2 bignerdmonster +1
    def __init__(self, cols=15, rows=11): 2 bignerdmonster +1
        ## initialize the matrix. kinda duh-doy type stuff, but important regardless.
        self.cols = cols
        self.rows = rows
        self.center = [(cols // 2), (rows // 2)]
        self.mat = [[0 for place in range(cols)] for row in range(rows)]
    def __str__(self): # for debugging of doom and gloom 2 bignerdmonster
        retStr = ""
        for row in self.mat:
            retStr += ' '.join([str(elem) for elem in row]) + "\n"
        return retStr.strip()

# i am dizzy. i will rest now. (ai told me to rest, i am "OBEYING" LAUGHING MY ASS OFF WTF) ### jacint
```

```
## create objects

if progress.contain("popup"):
    PopUps()

if progress.contain("music_box"):
    musicBox()

if keysPressed[ord('K') | ord('RETURN')]:
```

Progression

```
class Progression(): 1 usage 2 ToastPixel +1*
    def __init__(self): 2 ToastPixel +1*
        self.lvls = {
            10: "popup",
            14: "test",
            17: "music_box",
            25: "popup",
        }
        self.unlocked = []
        self.activated = []

    def check(self, length): 1 usage (1 dynamic) 2 ToastPixel
        newly_unlocked = []

        for n, f in self.lvls.items():
            if length >= n:
                if not self.unlocked.__contains__(f):
                    print(f"unlocked {f}")
                    self.unlocked.append(f)
                    self.activated.append(f)
                    newly_unlocked.append(f)
        return newly_unlocked

    def contain(self, name): 2 usages (2 dynamic) 2 ToastPixel
        if name in self.unlocked and name in self.activated:
            self.activated.remove(name)
            return True
        else:
            return False
```

How it works

- Dictionary of powerups
- Two lists
 - Once a certain length is reached it adds it to both lists
 - Once the contain method is called, it checks if it's inside both unlocked and activated and removes it from the activated list and returns a boolean
 - This ensures that it doesn't trigger twice
 - The reset method clears both lists so all of them can trigger again

Progression

```
if event.type == SNAKE_EVENT:
    GameObject.Collide(snake) # collide objects with snake before moving it
    ## create objects
    if progress.contain("popup"):
        PopUps()
    if progress.contain("music_box"):
        musicBox()
    if keyPressed[pg.K_RETURN]:
        Apple() # make apple.
    snake.move() # then move snake afterwards.
    if (random.randint(a: 0, b: 120) < 1) and (not QTE._instance) and ("QTE" in progress.unlocked):
        QTE()
```

How it works

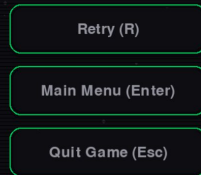
- Inside the main game loop, it calls the classes
- Ensures no second activation
- Since the classes call themselves again, there's no need to recall them

Q.O.L Features

- Restart system
- Sound and Visual Effects
- Re-rendering GameObjects
- Start and Gameover Menu + VFX
- Background/Screen progression
- Special Powerups (Golden and Poisonous apples)



GAME OVER!



class Menu:

```
def __init__(self, screenInp=None, title="Stake (get it like start and snake lol)", clocked=pg.time.Clock(), win_w=1080, win_h=720):
    self.notstop = True
    self.screen = screenInp or pg.display.set_mode((win_w, win_h), pg.SCALED, vsync=1)
    pg.display.set_caption(title)
    self.clock = clocked
    self.win_w = win_w
    self.win_h = win_h

    self.title_font = pg.font.SysFont(None, 110)
    self.small_font = pg.font.SysFont(None, 30)
    self.btn_font = pg.font.SysFont(None, 36)

    self.play_btn = Button("Play (Enter)", (self.win_w//2, self.win_h//2 + 40))
    self.howto_btn = Button("How to Play (H)", (self.win_w//2, self.win_h//2 + 140))
    self.quit_btn = Button("Quit (Esc)", (self.win_w//2, self.win_h//2 + 240))

    self.show_help = False
    self.lines = [
        "Use Arrow Keys or WASD to move.",
        "",
        "Press Esc to switch between this screen and the menu."
    ]

    self.starts = [(random.randint(0, win_w), random.randint(0, win_h), random.randint(1, 3)) for _ in range(120)]
    self.t = 0

    # Background snake
    self.snake_points = []
    self.snake_length = 60
    self.snake_dir = random.choice([
        pg.Vector2(4, 0),
        pg.Vector2(-4, 0),
        pg.Vector2(0, 4),
        pg.Vector2(0, -4)
    ])

    self.snake_pos = pg.Vector2(
        random.randint(100, self.win_w - 100),
        random.randint(100, self.win_h - 100)
    )
```

----- UI -----

You, 42 minutes ago | 3 authors (You and others)

class Button:

```
def __init__(self, txt, center, size=(320, 80)):
    self.txt = txt
    self.rect = pg.Rect(0, 0, *size)
    self.rect.center = center
```

```
def draw(self, surf, font=None):
```

```
    if not pg.font.get_init():
```

```
        pg.font.init()
```

```
    font = font or pg.font.SysFont(None, 36)
```

```
    hovering = self.rect.collidepoint(pg.mouse.get_pos())
```

```
    shadow_rect = self.rect.copy()
```

```
    shadow_rect.y += 6
```

```
    pg.draw.rect(surf, SHADOW, shadow_rect, border_radius=20)
```

```
    if hovering:
```

```
        glow = pg.Surface(self.rect.size, pg.SRCALPHA)
```

```
        glow.fill((*NEON, 90))
```

```
        surf.blit(glow, self.rect.topleft)
```

```
    pg.draw.rect(surf, ACCENT_H if hovering else ACCENT, self.rect, border_radius=20, width=2)
```

```
    inner = self.rect.inflate(-12, -12)
```

```
    pg.draw.rect(surf, BG, inner, border_radius=16)
```

```
    label = font.render(self.txt, True, TEXT if hovering else MUTED)
```

```
    surf.blit(label, label.get_rect(center=self.rect.center))
```

```
    return hovering
```

```
def clicked(self, event):
```

```
    return (
```

```
        event.type == pg.MOUSEBUTTONDOWN and event.button == 1
```

```
        and self.rect.collidepoint(event.pos)
```

```
)
```

```

def snakeGame(menu, snake, progress): ## this is the actual main game loop function!! yay
    best_score = 0
    run = True
    pg.mixer.music.load('images/background-music.mp3')
    pg.mixer.music.play(-1)
    while run:
        progress.check(snake.len)
        if snake.len < 5:
            screen.blit(bg1, (0, 0))
        elif snake.len < 10:
            screen.blit(bg2, (0, 0))
        elif snake.len < 15:
            screen.blit(bg3, (0, 0))
        elif snake.len < 20:
            screen.blit(bg4, (0, 0))
        else:
            screen.blit(bg5, (0, 0))

        drawGrid(screen)
        font = pg.font.SysFont(None, 36)
        if snake.len > best_score:
            best_score = snake.len

        score_str = "Score: " + str(snake.len)
        best_str = "Best Score: " + str(best_score)

        outline_color = (255, 255, 255)
        offsets = [(-1,0), (1,0), (0,-1), (0,1)]

        for ox, oy in offsets:
            screen.blit(font.render(score_str, True, outline_color), (5+ox, 10+oy))
            screen.blit(font.render(best_str, True, outline_color), (5+ox, 50+oy))

        score_text = font.render(score_str, True, (0,0,0))
        best_text = font.render(best_str, True, (0,0,0))

        screen.blit(score_text, (5, 10))
        screen.blit(best_text, (5, 50))

```

```

class PoisonApple(GameObject):
    def __init__(self, pos=None):
        super().__init__(pos or [random.randint(0, COLUMN_COUNT-1), random.randint(0, ROW_COUNT-1)])
        self.image = poison_apple_img

    def render(self, screenV):
        screenV.blit(self.image, self.rect)

    def collide(self, snake):
        snake.len = max(1, snake.len - 1)

        if SnakeTail.taillist:
            tail_piece = SnakeTail.taillist.pop()
            if tail_piece in GameObject.objList:
                GameObject.objList.remove(tail_piece)

        if self in GameObject.objList:
            GameObject.objList.remove(self)

        oof = pg.mixer.Sound('images/roblox-death-sound_1.mp3')
        oof.play()

        if random.random() < 0.2:
            PoisonApple()

```

AI Use

Menu Tweaks and VFX:

- ChatGPT added a lot of VFX... and it didn't work
- It did however set a lot of the groundwork for the menu basis
- It sets me back a couple times while working on it
- It had a lot of useful contributions, but it still needs to be reworked on.

```
def _draw_background_snake(self):
    SPEED = 4
    self.snake_pos += self.snake_dir * (SPEED / 8)

    if not hasattr(self, "turn_cooldown"):
        self.turn_cooldown = 0

    if self.turn_cooldown > 0:
        self.turn_cooldown -= 1
    else:
        if random.random() < 0.08:
            possible_dirs = [
                pg.Vector2(8, 0),
                pg.Vector2(-8, 0),
                pg.Vector2(0, 8),
                pg.Vector2(0, -8)
            ]

            reverse = -self.snake_dir
            valid_dirs = [d for d in possible_dirs if d != reverse]

            if valid_dirs:
                self.snake_dir = random.choice(valid_dirs)
                self.turn_cooldown = 8

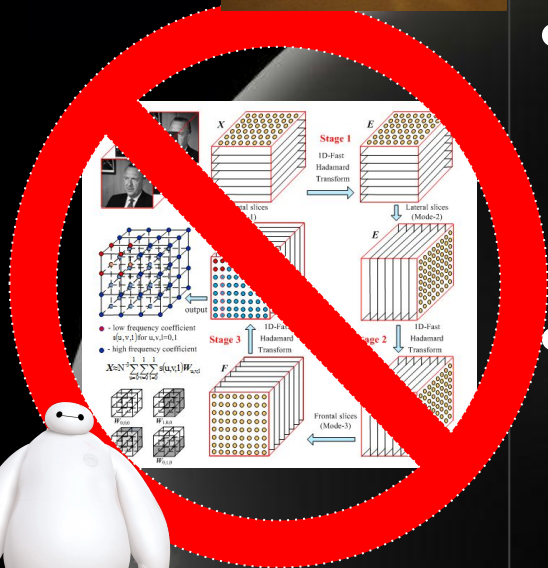
        if self.snake_pos.x < 0:
            self.snake_pos.x = self.win_w
        if self.snake_pos.x > self.win_w:
            self.snake_pos.x = 0
        if self.snake_pos.y < 0:
            self.snake_pos.y = self.win_h
        if self.snake_pos.y > self.win_h:
            self.snake_pos.y = 0

        self.snake_points.insert(0, (int(self.snake_pos.x), int(self.snake_pos.y)))
        if len(self.snake_points) > self.snake_length:
            self.snake_points.pop()

BLOCK_SIZE = 12
```

You, 16 hours ago • Menu revamp by PapaGPT

AI Use



- Google Gemini with Search
 - Genuinely the most helpful thing throughout this project. As a retired dl stack overflow sorter, I really appreciated the quick redirects towards the correct pages
- Microsoft Copilot with VS Code
 - I had autocomplete on while writing code, and it helped with repetitive tasks. Also kept me company, in a slightly ominous manner (it autocompleted "I am "OBEYING" at 3 am, which was hilarious to me)
- The Original Monoclass in Snake:
 - I wanted to make sure we didn't have two snakes, but didn't know how. Copilot told me, and then I used `_instance` (but not the rest, as we don't need to prevent multiple non-snakes from being created)

i am dizzy. i will rest now. (ai told me to rest, i am "OBEYING" LMAO WTF)

AI Use

Timers

- The idea of `pg.time.get_ticks()`
 - Allowed for the interactive features of music box and popups
 - Expanded the idea to create the ability to increase the timer by holding onto it, creating the music box class
- Timer separate from the core game loop

```
class Clickable(GameObject): 2 usages & Felix
    def __init__(self, pos=None): & Felix
        super().__init__(pos)
        self.last_tick = pg.time.get_ticks()
    def clicked(self): 3 usages & Felix
        return self.rect.collidepoint(pg.mouse.get_pos()) and (pg.mouse.get_pressed())[0]
```

```
class musicBox(Clickable): 5 usages & Felix +1
    maxTime = 1200
    def __init__(self, pos=[((COLUMN_COUNT//2)+(COLUMN_COUNT//4)),((ROW_COUNT//2)+(ROW_COUNT//4)),81]):
        super().__init__(pos)
        self.rect.scale_by_ip(9,7)
        self.time = musicBox.maxTime
        self.color = pg.color.Color(0, 0, 255, 76)
        #self.last_tick = pg.time.get_ticks() but in Clickable
    def render(self, screenV = screen): 1 usage (1 dynamic) & Felix +1
        #I need this to be translucent lmao
        current_time = pg.time.get_ticks()
        if (current_time - self.last_tick >= 1000) and not self.clicked():
            self.time -= 60 ## tick down by a second, only when not clicked tho
            self.last_tick = current_time
        super().render(screenV) # render box to screen

    text = font.render( text=f"Time: {self.time // 60}", antialias=True, color=(255, 255, 255))
    screenV.blit(text, dest=(self.rect.x + 10, self.rect.y + 15)) # render text to screen

    if self.clicked() and self.time < musicBox.maxTime:
        self.time += 2 #slow wind.
```

Reflection



Some of the biggest challenges were finding out how to use our new tools

- Git (!!!)
- Pygame

Debugging and finding out what was wrong with our code

- Improper layering
- Time management

Reflections



- Using / learning how to use GitHub while coding with others was difficult
- ChatGPT had a lot of good ideas, but not the best executions of those ideas
- "Re-inventing the wheel" happened A LOT
- It was a lot of fun to work on a project with others in this way

Reflection

My biggest issue in this project was with time management. I enjoy programming a lot, but I found that I worked in shorter, ~3 day long bursts of passion instead of consistently over the whole assignment timeline. While this left me eager to work on the project and always had me excited to implement something new, it meant that my group mates had to deal with their code being made obsolete or otherwise referring to stuff that didn't exist. This wouldn't work in a professional setting, and for future projects, I plan on working with a schedule.

Concerning the game itself, I'm genuinely proud of its modularity, and I think it could be expanded further without much work (maybe that's a consequence of the problem above, though...). I plan on actually going back and playing around with more settings after we submit, as I want to see how far I can go...

As an aside, being the only member of the group with prior `git` experience led to some silly situations... but that was fun.

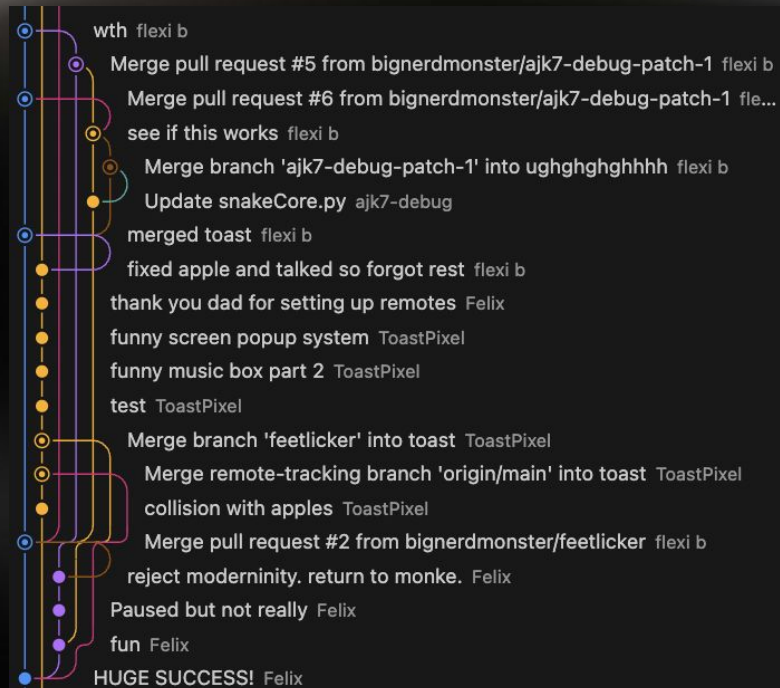
Git History

Tragedy.

```

• idk what I touched bignermonster
• bg 2 returned ToastPixel
• Crossant Felix
• fixed the Felix
• Working QTE mod Felix
• Fixed background Felix
• Merge branch 'toast' of https://github.com/bignermonster/snakeCE160...
  • whoops ToastPixel
  • Merge toast Felix
  • yogurt Felix
  • Merge remote-tracking branch 'origin/toast' into toast ToastPixel
  • "QTE stuff" ToastPixel
  • Merge toast into main Felix
  • revert some changes Felix
  • doesn't matter Felix
  • uhh where's the snake head Felix
  • Merged toast, mango, ajk7-debug-2, life, the universe, and everything...
  • Just a bit really Felix
  • damn arasaas not getting me this time Felix
  • No more ignoring the popup Felix
  • rearrange constants, add golden apple Felix
  • did a lot Felix
  • toast Felix
  • refactor? hardly Felix
  • - ToastPixel
  • a bit of edits ToastPixel
  • idk what I did Felix
  • idk what I did Felix
  • stuff, Felix
  • uhh, Felix
  • Add files via upload ajk7-debug
  • Update snakeCore.py ajk7-debug
  • Update snakeCore.py ajk7-debug
  • Add files via upload ajk7-debug
  • Add files via upload ajk7-debug
  • merge one toast Felix
  • I don't know what I did, but I fixed the project Felix
  • uhh? Felix
  • WHEN DID APPLE COLLISION BREAK, Felix
  • pc almost dead flexi b
  • mango? hardly know at flexi b
  • Merge toast into main, working code only flexi b
  • uhhh idk what I changed flexi b
  • Merge branch 'toast' of https://github.com/bignermonster/snakeCE16...
  • minor changes flexi b
  • test ToastPixel
  • holy merge ToastPixel
  • uhhhh ToastPixel
  • Merged Alan's progression logic, but like... got it working with Git Toast...
  • what if it fully worked ToastPixel
  • Merge remote-tracking branch 'origin/toast' into toast ToastPixel
  • removed redundant code and also yap flexi b
  • added ram name for funny flexi b
  • merge snake core changes flexi b
  • reject snake mat, its useless and matlines are for fools, flexi b
  • stuff I didn't commit to toast before branching flexi b
  • documentation flexi b
  • Added Reset fn for classes, made popup render last in GameObject...
  • - ToastPixel
  • changed popup more flexi b
  • Music box translated to core flexi b
  • with flexi b
  • Merge pull request #5 from bignermonster/ajk7-debug-patch-1 flexi b
  • Merge pull request #6 from bignermonster/ajk7-debug-patch-1 flexi b
  • see if this works flexi b
  • Merge branch 'ajk7-debug-patch-1' into ughghghghhhh flexi b
  • Update snakeCore.py ajk7-debug
  • merged toast flexi b
  • fixed apple and talked so forgot rest flexi b
  • thank you dad for setting up remotes Felix
  • funny screen popup system ToastPixel
  • funny music box part 2 ToastPixel
  • test ToastPixel
  • Merge branch 'feetlicker' into toast ToastPixel
  • Merge remote-tracking branch 'origin/main' into toast ToastPixel
  • collision with apples ToastPixel
  • Merge pull request #2 from bignermonster/feetlicker flexi b
  • reject modernity, return to monke, Felix
  • Paused but not really Felix
  • fun Felix
  • HUGE SUCCESS! Felix
  • mostly working -- hey peter watch me remove typing Felix
  • Merged Feetlicker, adding a rendering system for SnakeMat, Snake, Apple, ...
  • added WLD jerk Felix
  • test 2 Felix
  • testing signed commit + render engine begin FELIX

```



Thank you